L3: Lab Report-MPI

1 Experiment Setup

Configuration	Configuration instructions	
Hardware configuration	CPU	Intel(R) Xeon(R) Gold 6330 CPU @ 2.00GHz
	Memory	256 GiB
	Storage	1.8 TB local disk (47 GB available), 57 TB network storage (Lustre filesystem)
Software configuration	Operating System	Ubuntu 20.04 (Linux kernel 5.4.0-208-ge neric)
	Compiler	g++ 9.4.0
	MPI version	mpirun (Open MPI) 4.0.3

2. Performance Evaluation of Conjugate Gradient

2.1 Performance of Sequential Conjugate Gradient

Implementation Details:

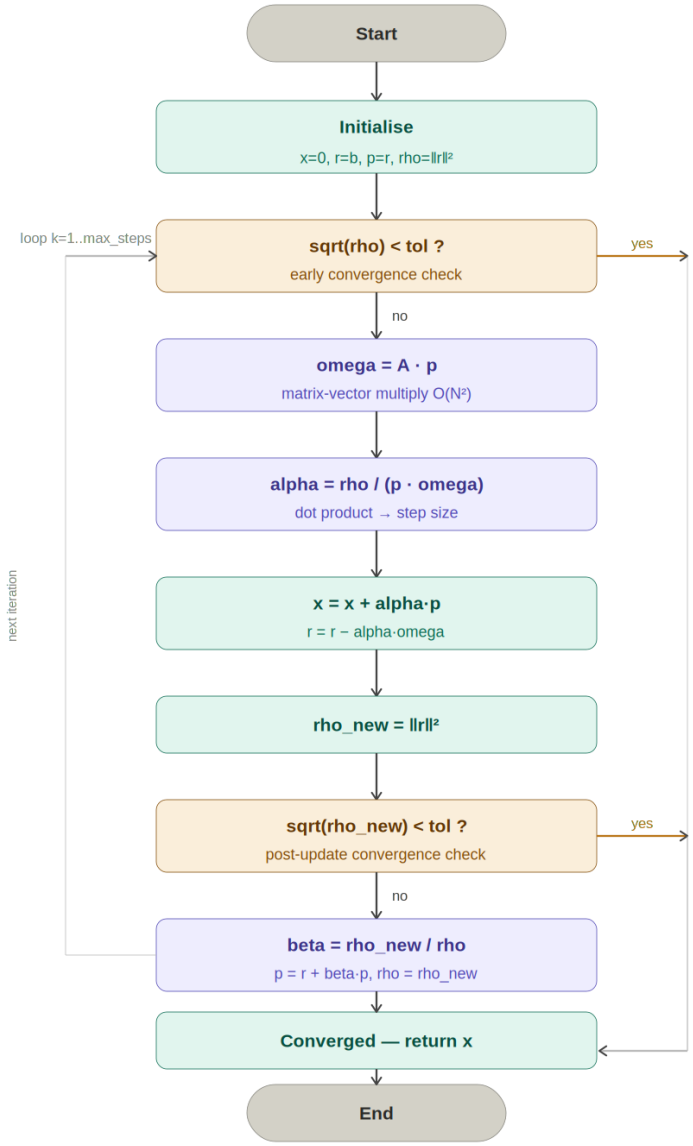
The serial Conjugate Gradient (CG) implementation solves the symmetric positive definite linear system (Ax=b) iteratively. The algorithm first allocates memory for the solution vector (x), residual vector (r), search direction vector (p), and temporary vector ω. The initial solution is set to zero, the residual is initialized as (r=b), and the search direction is set to (p=r). The squared residual norm (ρ=r^Tr) is then computed. During each iteration, the matrix-vector multiplication (ω=Ap) is performed using nested loops

over the dense matrix. The step size (α) is calculated as (ρ/(p^T ω)), after which the solution vector and residual vector are updated using (x=x+ αp) and (r=r- αω). The new residual norm (ρnew = r^Tr) is computed and compared against the convergence tolerance. If convergence has not been reached, the coefficient (β = ρnew / ρ) is calculated and the search direction is updated as (p=r+βp). The process repeats until either the maximum number of iterations is reached or the residual norm falls below the specified tolerance. Finally, all temporary memory is released and the approximated solution vector is returned.

Pseudocode:

```
Input:   A (N×N SPD), b (N), max_steps, tol
x=0;   r=b;   p=r;   rho=||r||²
for k = 1 to max_steps:
      if sqrt(rho) < tol: break
      omega    = A * p
      alpha    = rho / (p^T * omega)
      x        = x + alpha * p
      r        = r - alpha * omega
      rho_new = ||r||²
      if sqrt(rho_new) < tol: break
      beta     = rho_new / rho
      p        = r + beta * p
      rho      = rho_new
```

Flow Chart:



Results:

Matrix Dimension	Step Size	Execution Time (s)
1024	50	0.012412
	100	0.012389
	200	0.012394
	500	0.012397

2048	50	0.051631
	100	0.051363
	200	0.051338
	500	0.051658
4096	50	0.239446
	100	0.238972
	200	0.238996
	500	0.238898
8192	50	0.901125
	100	0.893356
	200	0.890121
	500	0.879744

The results show that execution time increases significantly as the matrix dimension grows from 1024 to 8192. This is expected because larger matrices require more computations during the matrix-vector multiplication step, which is the most expensive operation in the Conjugate Gradient algorithm.

In contrast, changing the maximum step size from 50 to 500 has almost no effect on execution time. This indicates that the solver converges before reaching the maximum number of iterations, so increasing the step limit does not increase the actual amount of work performed.

Overall, matrix size is the main factor affecting execution time, while the maximum step size has minimal impact on performance.

2.2 Performance of Parallel Conjugate Gradient

Implementation Details:

The parallel implementation distributes the matrix rows and vectors among multiple MPI processes using a one-dimensional row-wise decomposition. Each process stores only a subset of the matrix rows and the corresponding vector elements. Similar to the serial version, the local solution vector is initialized to zero, the local residual is set to the local portion of b, and the search direction is initialized from the residual. Each process computes its local contribution to the residual norm, and a global residual norm is obtained using MPI_Allreduce.

During every CG iteration, the matrix-vector multiplication Ap is performed cooperatively by all processes. Since each process owns only part of the search vector p, a ring communication pattern is used where processes exchange portions of p with neighboring processes using MPI_Isend and MPI_Recv. As each block of p arrives, the process multiplies it with the corresponding columns of its local matrix rows and accumulates the result in ω. Once the distributed matrix-vector multiplication is completed, each process computes its local contribution to the dot product p^Tω, and MPI_Allreduce is used to obtain the global value required to calculate α. The local solution and residual vectors are then updated. The new residual norm is again computed collectively using MPI_Allreduce, allowing all processes to determine convergence simultaneously. If convergence is not achieved, the search direction is updated locally and the next iteration begins. At the end of the computation, each process holds its portion of the approximated solution vector.

Problem Partitioning:

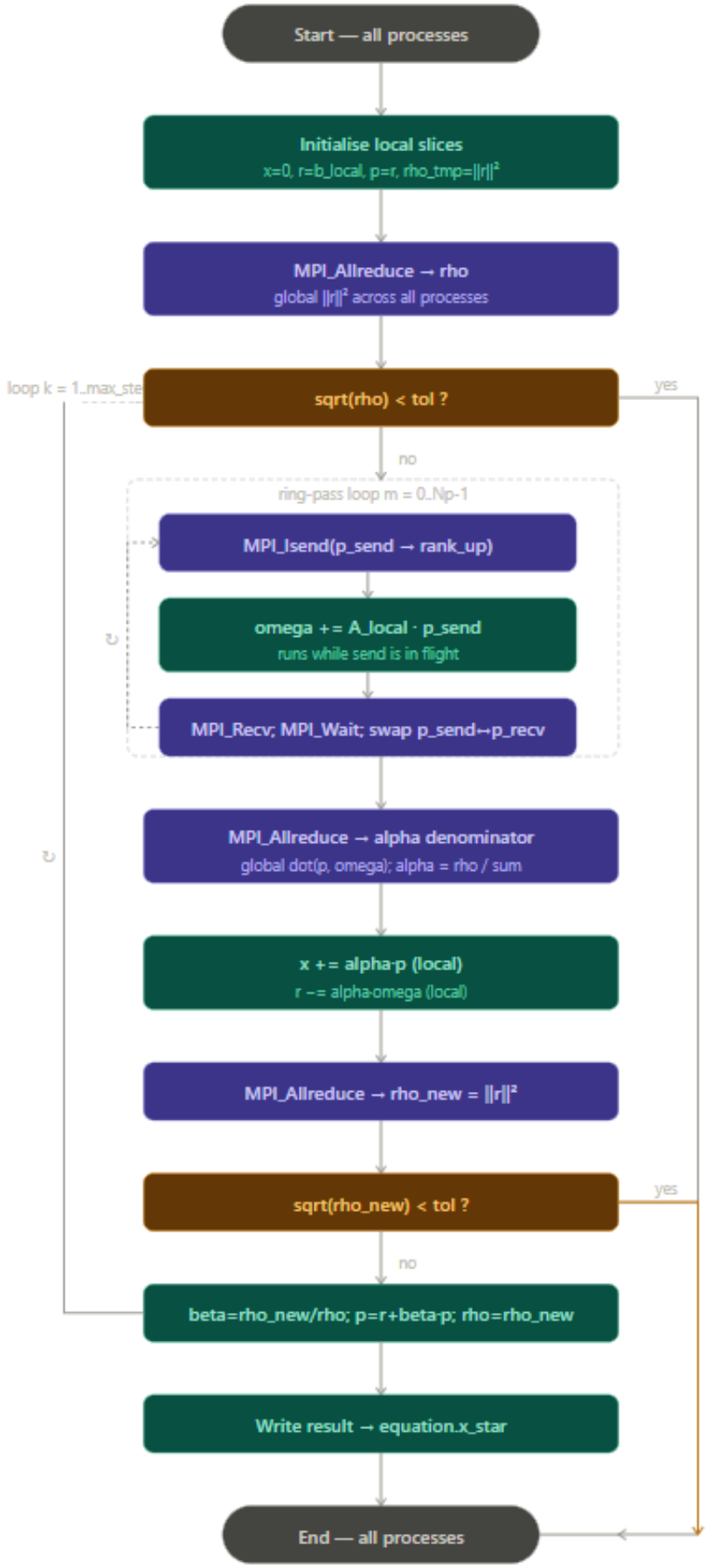
The problem is partitioned using row-wise domain decomposition. The N×N matrix is divided into contiguous blocks of rows, and each MPI process is assigned

approximately N/Np rows, where Np is the number of processes. The corresponding portions of vectors b, x, r, p, and ω are stored locally by each process. This decomposition reduces memory requirements because no process needs to store the entire matrix or vectors. The workload is balanced by distributing rows as evenly as possible among all processes.

For example, for N=8192 and Np=4, each process owns 2048 consecutive rows of A, the matching slice of b, and writes its result into equation.x_star.

```
Full N×N matrix A              Distributed to Np=4 processes
+------------------+
| rows  0 .. 2047  |  --> Process 0:  A[0..2047,  0..N]
+------------------+
| rows 2048 .. 4095|  --> Process 1:  A[2048..4095, 0..N]
+------------------+
| rows 4096 .. 6143|  --> Process 2:  A[4096..6143, 0..N]
+------------------+
| rows 6144 .. 8191|  --> Process 3:  A[6144..8191, 0..N]
+------------------+
```

Flow Chart:


Start — all processes
Initialise local slices
x=0, r=b_local, p=r, rho_tmp=||r||²
MPI_Allreduce → rho
global ||r||² across all processes
loop k = 1..max_ste
sqrt(rho) < tol ?
yes
no
ring-pass loop m = 0..Np-1
MPI_Isend(p_send → rank_up)
omega += A_local · p_send
runs while send is in flight
MPI_Recv; MPI_Wait; swap p_send↔p_recv
MPI_Allreduce → alpha denominator
global dot(p, omega); alpha = rho / sum
x += alpha·p (local)
r −= alpha·omega (local)
MPI_Allreduce → rho_new = ||r||²
sqrt(rho_new) < tol ?
yes
no
beta=rho_new/rho; p=r+beta·p; rho=rho_new
Write result → equation.x_star
End — all processes

Parallel Features Used:

- MPI Cartesian communicator: MPI_Cart_create() builds a 1D virtual topology. Each process gets a coordinate that determines which rows of A it owns, simplifying neighbour addressing (rank_up, rank_down).
- Non-blocking send (MPI_Isend) : In the ring-passing loop, MPI_Isend posts the send before local computation begins. The message travels over the network while the CPU accumulates omega, meaning communication is hidden behind computation.
- Point-to-point communication (MPI_Recv): Used to receive vector segments during the ring exchange.
- Collective communication (MPI_Allreduce): Two MPI_Allreduce calls per iteration (MPI_SUM) gather global sums from all processes: one for rho = global ||r||^2 and one for the alpha denominator = global dot(p, omega). MPI_Allreduce ensures every process gets the result and can continue without a broadcast.
- MPI_Barrier: Used to perform synchronization is performed to ensure fair timing measurements across all processes.
- MPI_Wtime: Used to measure timing, which provides high-resolution wall-clock timing for MPI programs.

Optimizations Used:

First, the matrix is distributed across processes, reducing the memory footprint per process and allowing larger problem sizes to be solved. Second, a ring-based communication strategy is used for the matrix-vector multiplication, avoiding the need to broadcast the entire search vector to every process. Third, non-blocking communication (MPI_Isend) allows communication to overlap partially with computation, reducing communication overhead. Fourth, only the necessary global reductions are performed using MPI_Allreduce, minimizing synchronization costs while ensuring correctness. Finally, the code is compiled with the -O3 optimization flag,

enabling compiler-level optimizations such as loop optimization and instruction-level improvements to enhance execution performance.

Results:

Matrix Size = 1024

Step Size	Process Number	Execution Time (s)	Speed Up
50	1	0.012361	1.00412588
	2	0.006126	2.026118185
	4	0.004014	3.092177379
	8	0.002507	4.950937375
	16	0.002312	5.368512111
	32	0.001112	11.1618705
	64	0.001357	9.146647015
100	1	0.012339	1.004052192
	2	0.006130	2.021044046
	4	0.003980	3.11281407
	8	0.002204	5.621143376
	16	0.001850	6.696756757
	32	0.001451	8.538249483
	64	0.002293	5.402965547
200	1	0.012338	1.004538823
	2	0.006117	2.026156613
	4	0.003188	3.88770389
	8	0.002027	6.114454859
	16	0.001521	8.148586456
	32	0.001207	10.26843413
	64	0.001984	6.246975806
500	1	0.012333	1.005189329

	2	0.006112	2.028304974
	4	0.004158	2.981481481
	8	0.002618	4.735294118
	16	0.001738	7.132911392
	32	0.001437	8.627000696
	64	0.001888	6.566207627

Matrix Size = 2048

Step Size	Process Number	Execution Time (s)	Speed Up
50	1	0.050907	1.014222013
	2	0.025467	2.027368752
	4	0.012273	4.206876884
	8	0.005690	9.073989455
	16	0.003457	14.93520393
	32	0.002000	25.8155
	64	0.003914	13.19136433
100	1	0.050829	1.010505814
	2	0.025466	2.016924527
	4	0.012323	4.168059726
	8	0.005544	9.26461039
	16	0.003345	15.35515695
	32	0.002400	21.40125
	64	0.002971	17.28811848
200	1	0.050884	1.008922255
	2	0.025464	2.016101162
	4	0.012280	4.180618893
	8	0.005543	9.261771604
	16	0.003236	15.86464771
	32	0.002094	24.51671442

	64	0.002271	22.60590048
500	1	0.050808	1.016729649
	2	0.025454	2.029464917
	4	0.012296	4.201203643
	8	0.005621	9.190179683
	16	0.003164	16.32680152
	32	0.003125	16.53056
	64	0.002531	20.41011458

Matrix Size = 4096

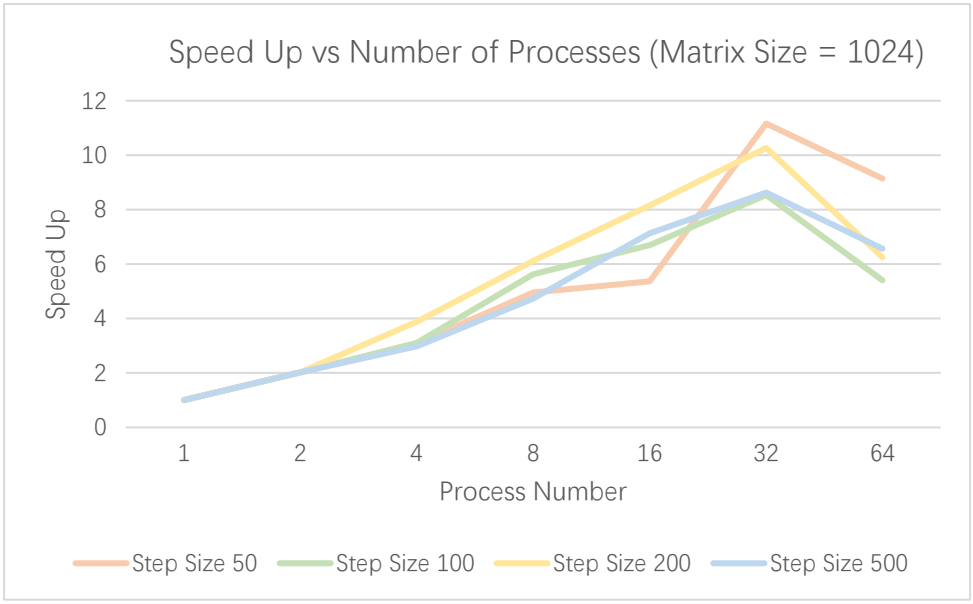
Step Size	Process Number	Execution Time (s)	Speed Up
50	1	0.211046	1.134567819
	2	0.107606	2.22521049
	4	0.053304	4.492083146
	8	0.027479	8.713781433
	16	0.015389	15.55955553
	32	0.008379	28.57691849
	64	0.010357	23.11924302
100	1	0.210723	1.134057507
	2	0.107709	2.218681819
	4	0.053435	4.472199869
	8	0.027395	8.723197664
	16	0.015381	15.53683116
	32	0.008475	28.19728614
	64	0.011166	21.40175533
200	1	0.210815	1.133676446
	2	0.107627	2.220595204
	4	0.053661	4.453811893
	8	0.027403	8.72152684

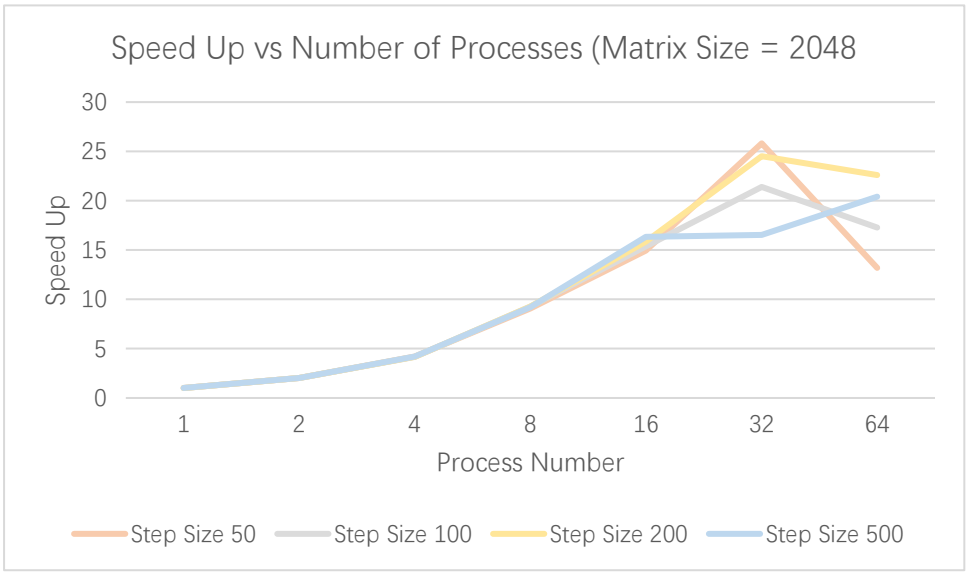
	16	0.015512	15.40716864
	32	0.008502	28.11056222
	64	0.010697	22.34233897
500	1	0.210820	1.133184707
	2	0.107755	2.217047933
	4	0.053491	4.46613449
	8	0.027421	8.712227855
	16	0.015397	15.51587972
	32	0.008432	28.3323055
	64	0.010234	23.34356068

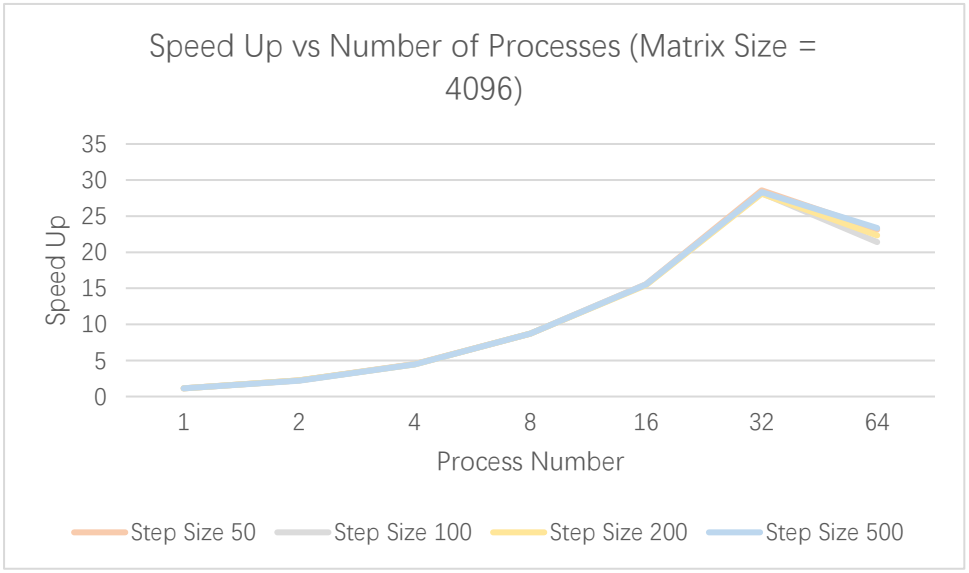
Matrix Size = 8192

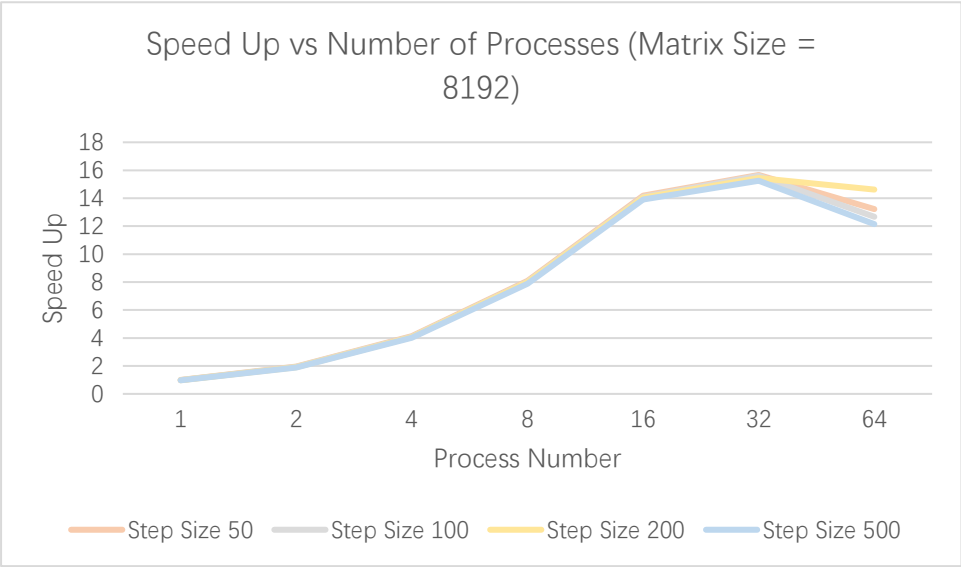
Step Size	Process Number	Execution Time (s)	Speed Up
50	1	0.904768	0.995973553
	2	0.461279	1.953535713
	4	0.218579	4.122651307
	8	0.111515	8.080751468
	16	0.063541	14.18178814
	32	0.057533	15.66275007
	64	0.068135	13.22558157
100	1	0.904690	0.987471952
	2	0.460664	1.939278954
	4	0.218456	4.089409309
	8	0.111493	8.012664472
	16	0.063387	14.09367851
	32	0.057248	15.60501677
	64	0.070488	12.67387357
200	1	0.908921	0.979316134
	2	0.465305	1.912983957

	4	0.218734	4.069422221
	8	0.111385	7.991390223
	16	0.063413	14.03688518
	32	0.057643	15.44196173
	64	0.060921	14.61107007
500	1	0.904624	0.972496861
	2	0.465306	1.890678392
	4	0.218774	4.021245669
	8	0.111417	7.895958426
	16	0.063227	13.9140557
	32	0.057690	15.24950598
	64	0.072376	12.15518957


Speed Up vs Number of Processes (Matrix Size = 1024)
Speed Up
12
10
8
6
4
2
0
1
2
4
8
16
32
64
Process Number
Step Size 50
Step Size 100
Step Size 200
Step Size 500


Speed Up vs Number of Processes (Matrix Size = 2048
Speed Up
30
25
20
15
10
5
0
1
2
4
8
16
32
64
Process Number
Step Size 50
Step Size 100
Step Size 200
Step Size 500


Speed Up vs Number of Processes (Matrix Size = 4096)
Speed Up
35
30
25
20
15
10
5
0
1
2
4
8
16
32
64
Process Number
Step Size 50
Step Size 100
Step Size 200
Step Size 500



The results show that the parallel Conjugate Gradient implementation achieves significant performance improvement as the number of processes increases. For all matrix sizes, the execution time decreases and the speedup increases when moving from 1 process to 32 processes. This indicates that the workload is effectively distributed among the MPI processes and that the parallelization strategy successfully reduces the computation time.

For larger matrices (2048, 4096, and 8192), the speedup scales almost linearly up to 32 processes. The best performance is observed for the 4096×4096 matrix, which achieves a speedup of approximately 28× when using 32 processes. Similarly, the 2048×2048 matrix reaches a speedup of around 25×, while the 8192×8192 matrix achieves a speedup of about 15×. These results demonstrate that larger computational workloads benefit more from parallel execution because the communication overhead becomes relatively smaller compared to the computation cost.

However, when the number of processes increases from 32 to 64, the speedup either decreases or shows only a small improvement. This behavior can be seen clearly in both the tables and line charts. The reason is that communication and synchronization overheads become more significant at higher process counts. Operations such as MPI_Allreduce, message passing during the ring communication for matrix-vector multiplication, and process synchronization introduce additional costs that limit scalability.

The effect of the step size is minimal across all experiments. The speedup curves for step sizes 50, 100, 200, and 500 are very similar, indicating that the convergence behavior of the Conjugate Gradient algorithm is largely independent of the maximum step limit. Since the solver typically converges before reaching the maximum number of iterations, increasing the step size does not significantly affect the execution time or speedup.

Overall, the parallel implementation demonstrates good scalability and efficiency up to 32 processes. Beyond this point, communication overhead begins to dominate, causing diminishing returns in performance improvement. The results confirm that MPI-based parallelization effectively accelerates the Conjugate Gradient solver, particularly for larger matrix dimensions.

3.1 Performance of Sequential N-body

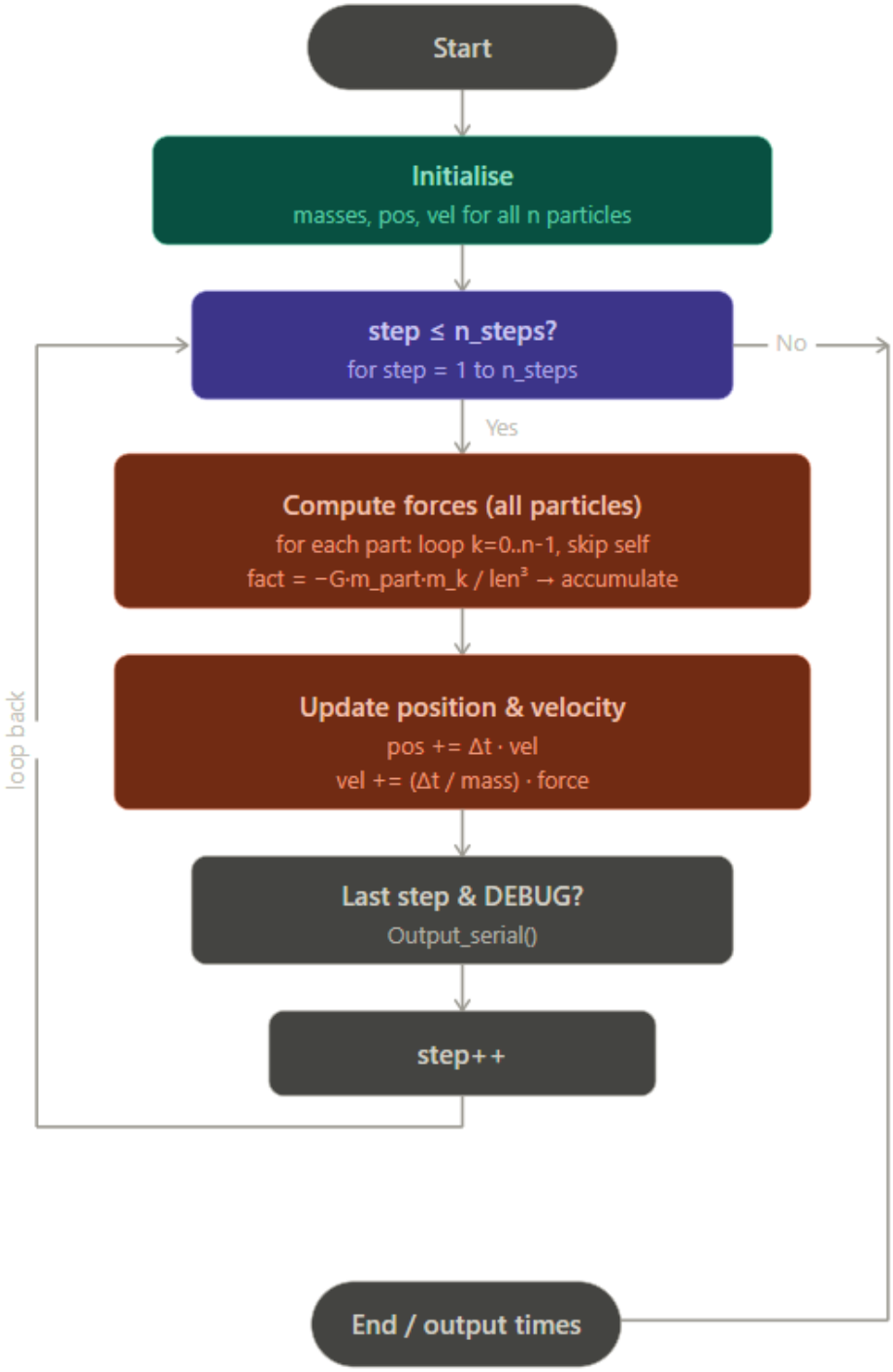
3.1.1 Implementation Details

In the serial implementation, the program simulates the motion of n particles over a number of time steps by repeatedly computing gravitational interactions and updating particle states. At each timestep, it first computes the net force on every particle using a double loop over all particle pairs, applying Newton's law of gravitation to calculate the pairwise force and summing contributions in both x and y directions. This results in an $O(n^2)$ force computation phase. After all forces are computed, the program updates each particle's velocity and position using a simple time-stepping scheme: the velocity is updated based on acceleration (force divided by mass), and the position is updated using the updated velocity and timestep size Δt. This process is repeated sequentially for all timesteps, producing the final positions and velocities.

Pseudocode:

```
// runs on rank 0 only
for step = 1 to n_steps:

   // Phase 1: forces
   for part = 0 to n−1:
      forces[part] = {0, 0}
      for k = 0 to n−1:
         if k == part: skip
         dx = pos[part].x − pos[k].x
         dy = pos[part].y − pos[k].y
         len = sqrt(dx²+dy²)
         fact = −G·m_p·m_k / len³
         forces[part] += fact·(dx,dy)

   // Phase 2: update
   for part = 0 to n−1:
      pos[part] += Δt · vel[part]
      vel[part] += (Δt/m) · forces[part]

   if DEBUG && last step:
      Output_serial()
```

Flow Chart:


Start
Initialise
masses, pos, vel for all n particles
step ≤ n_steps?
for step = 1 to n_steps
No
Yes
Compute forces (all particles)
for each part: loop k=0..n-1, skip self
fact = −G·m_part·m_k / len³ → accumulate
Update position & velocity
pos += Δt · vel
vel += (Δt / mass) · force
Last step & DEBUG?
Output_serial()
step++
loop back
End / output times

3.1.2 Performance Results

Number of Particles	Time-step Length	Number of Time-steps	Execution Time
128	1	50	0.005094
		100	0.007243
		200	0.013305
		500	0.033319
256	1	50	0.013235
		100	0.026459
		200	0.052850
		500	0.132108
512	1	50	0.052713
		100	0.105283
		200	0.210925
		500	0.527627
1024	1	50	0.210082
		100	0.421049
		200	0.842046
		500	2.105209

The serial implementation results show that the execution time increases steadily as both the number of particles and the number of timesteps increase. For a fixed number of particles, doubling the number of timesteps approximately doubles the execution time. For example, with 1024 particles, the runtime increases from 0.210 seconds for 50 timesteps to 0.421 seconds for 100 timesteps, 0.842 seconds for 200 timesteps, and 2.105 seconds for 500 timesteps. This indicates that the simulation performs a similar amount of work during each timestep, resulting in a nearly linear relationship between execution time and the number of timesteps.

The results also show that the execution time grows much faster as the number of particles increases. For instance, at 100 timesteps, increasing the particle count from 128 to 256 approximately quadruples the runtime from 0.007 seconds to 0.026 seconds, while increasing from 256 to 512 and from 512 to 1024 produces a similar fourfold increase. This behavior is expected because the force computation requires every

particle to interact with every other particle, giving the algorithm a time complexity of $O(n^2)$. Therefore, doubling the number of particles results in roughly four times more force calculations.

Overall, the serial results confirm the expected computational behavior of the n-body simulation. The runtime scales linearly with the number of timesteps and quadratically with the number of particles, demonstrating that the pairwise force calculation is the dominant cost in the simulation. This increasing computational cost is the primary motivation for using MPI parallelization to distribute the workload across multiple processes and reduce execution time.

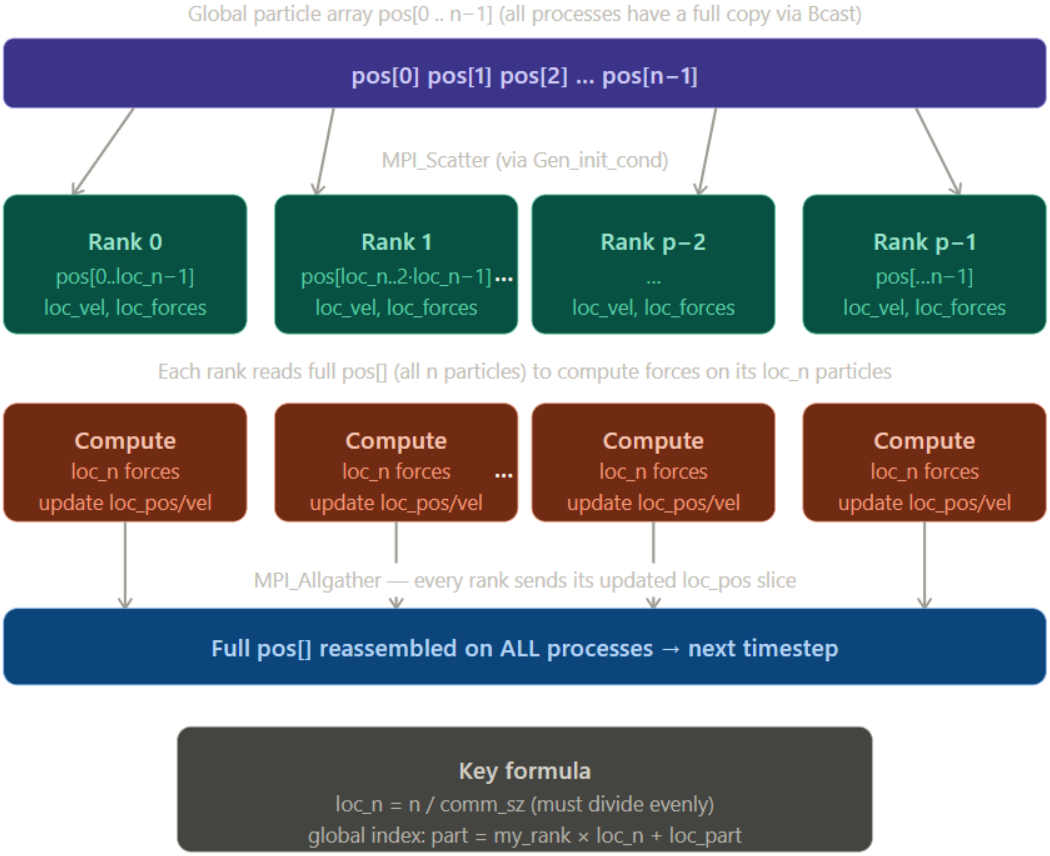
3.2 Performance of Parallel N-body

3.2.1 Implementation Details

In the parallel implementation, the particles are divided among MPI processes using a block decomposition, where each process is responsible for a contiguous subset of particles determined by its rank. Each process computes forces and updates only its local particles, reducing the computational workload per process from $O(n^2)$ to approximately $O(n^2/p)$. However, because gravitational force depends on all particles, each process still requires global position information. Therefore, after each timestep, an MPI_Allgather operation is used to synchronize updated positions across all processes, ensuring consistency for the next force computation.

Partitioning:

The n particles are divided into p equal blocks using a 1-D block distribution, where each process owns loc_n = n / p consecutive particles. Process rank r is responsible for global particle indices r * loc_n to (r+1) * loc_n - 1. This mapping is computed on the fly inside the loops as part = my_rank * loc_n + loc_part. The initial velocities are distributed to each process using MPI_Scatter, while masses and positions are broadcast to all processes using MPI_Bcast since every process needs the full arrays to compute forces correctly.



Parallel Features Used:

- MPI_Bcast shares the initial masses and position arrays to all processes, and also distributes the program arguments (n, n_steps, delta_t).
- MPI_Scatter distributes the initial velocity array so each process receives exactly its loc_n velocities.
- MPI_Allgather is invoked at the end of every timestep to collect each process's updated local positions and reassemble the complete pos[] array on all processes, ensuring correct force computation in the next step.
- MPI_Gather is used inside the debug output function to collect velocities back to rank 0 for printing.
- MPI_Barrier is placed before and after the parallel simulation to ensure all processes start and finish together, making the MPI_Wtime measurement on rank 0 an accurate wall-clock time for the full parallel execution.

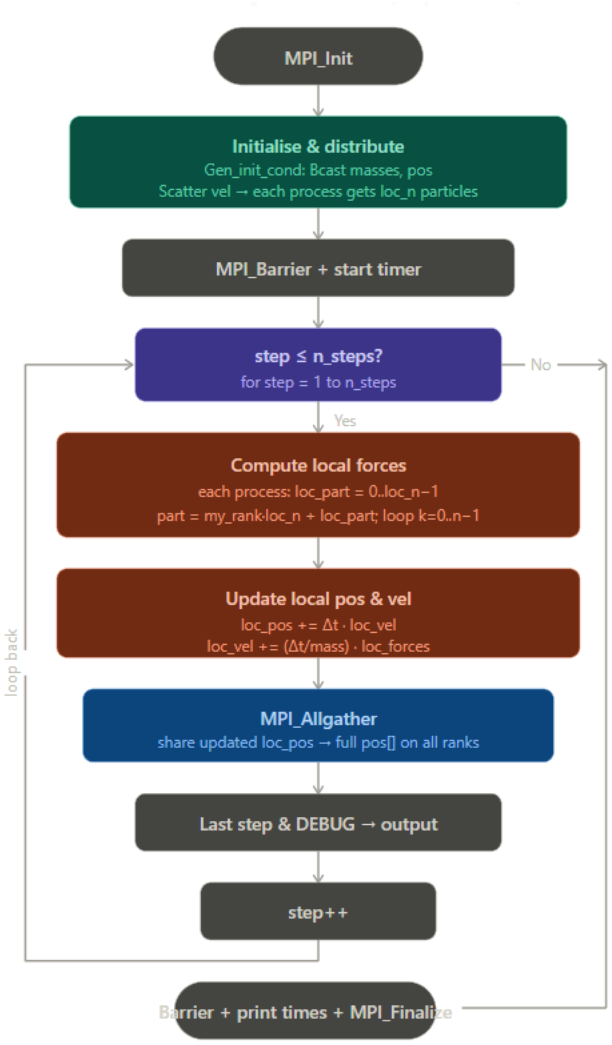
Optimizations:

Three optimisations are applied. First, loc_pos is not a separate array but a pointer directly into the correct slice of the global pos[] array (loc_pos = pos + my_rank * loc_n), so writing to loc_pos automatically updates pos[] in-place with no extra copy, and MPI_Allgather can gather directly into the right positions. Second, a derived MPI datatype vect_mpi_t is created using MPI_Type_contiguous to represent the 2-element double vector, allowing the 2-D position and velocity vectors to be sent in a single MPI operation rather than two separate scalar transfers. Third, the Makefile compiles with -O3, which enables aggressive compiler optimisations including loop unrolling and auto-vectorisation on the inner force computation loop, which is the hottest code path at $O(n^2)$ per timestep.

Pseudocode:

```
MPI_Barrier()   // sync timer
for step = 1 to n_steps:

  // Phase 1: local forces only
  for loc_part = 0 to loc_n−1:
    part = my_rank·loc_n + loc_part
    loc_forces[loc_part] = {0,0}
    for k = 0 to n−1:
      if k == part: skip
      // same Newton formula
      loc_forces[loc_part] += ···

  // Phase 2: local update
  for loc_part = 0 to loc_n−1:
    part = my_rank·loc_n + loc_part
    loc_pos[loc_part] += Δt · loc_vel[loc_part]
    loc_vel[loc_part] += (Δt/m) · loc_forces[loc_part]

  MPI_Allgather(loc_pos → pos)

  if DEBUG && last step:
    Output_parallel()
MPI_Barrier()   // stop timer
```

Flow Chart:


MPI_Init
Initialise & distribute
Gen_init_cond: Bcast masses, pos
Scatter vel → each process gets loc_n particles
MPI_Barrier + start timer
step ≤ n_steps?
for step = 1 to n_steps
No
Yes
Compute local forces
each process: loc_part = 0..loc_n−1
part = my_rank·loc_n + loc_part; loop k=0..n−1
Update local pos & vel
loc_pos += Δt · loc_vel
loc_vel += (Δt/mass) · loc_forces
MPI_Allgather
share updated loc_pos → full pos[] on all ranks
Last step & DEBUG → output
step++
loop back
Barrier + print times + MPI_Finalize

3.2.2 Performance Results

Number of Particles: 128

Number of time-steps	Time-step Length	Process Number	Execution Time	Speed Up
50	1	1	0.005083	1.002164076
		2	0.003669	1.388389207
		4	0.002842	1.792399719
		8	0.002002	2.544455544
		16	0.001499	3.39826551
		32	0.001573	3.238397966
		64	0.003689	1.380862022
100	1	1	0.009128	0.79349255
		2	0.006304	1.148953046
		4	0.006154	1.176958076
		8	0.003651	1.983840044
		16	0.002551	2.839278714
		32	0.002463	2.940722696
		64	0.006782	1.067974049
200	1	1	0.016833	0.790411691
		2	0.010808	1.231032568
		4	0.006331	2.101563734
		8	0.005611	2.371235074
		16	0.004919	2.704818052
		32	0.004183	3.180731532
		64	0.008844	1.504409769
500	1	1	0.035891	0.928338581
		2	0.021341	1.561267045
		4	0.013599	2.450106625

		8	0.010470	3.182330468
		16	0.010160	3.279429134
		32	0.008544	3.899695693
		64	0.025042	1.330524718

Number of Particles: 256

Number of time-steps	Time-step Length	Process Number	Execution Time	Speed Up
50	1	1	0.015730	0.841385887
		2	0.010582	1.250708751
		4	0.010556	1.253789314
		8	0.005926	2.233378333
		16	0.003350	3.950746269
		32	0.002676	4.945814649
		64	0.004893	2.704884529
100	1	1	0.029971	0.882820059
		2	0.016466	1.606886918
		4	0.014268	1.854429493
		8	0.010171	2.60141579
		16	0.006771	3.90769458
		32	0.004671	5.664525797
		64	0.007388	3.581348132
200	1	1	0.055230	0.956907478
		2	0.030924	1.709028586
		4	0.023976	2.204287621
		8	0.017198	3.073031748
		16	0.011412	4.631090081
		32	0.008232	6.420068027
		64	0.011252	4.696942766

500	1	1	0.135619	0.974111297
		2	0.073093	1.807396057
		4	0.040579	3.255575544
		8	0.027201	4.856733208
		16	0.020930	6.311896799
		32	0.014426	9.157632053
		64	0.027795	4.752941176

Number of Particles: 512

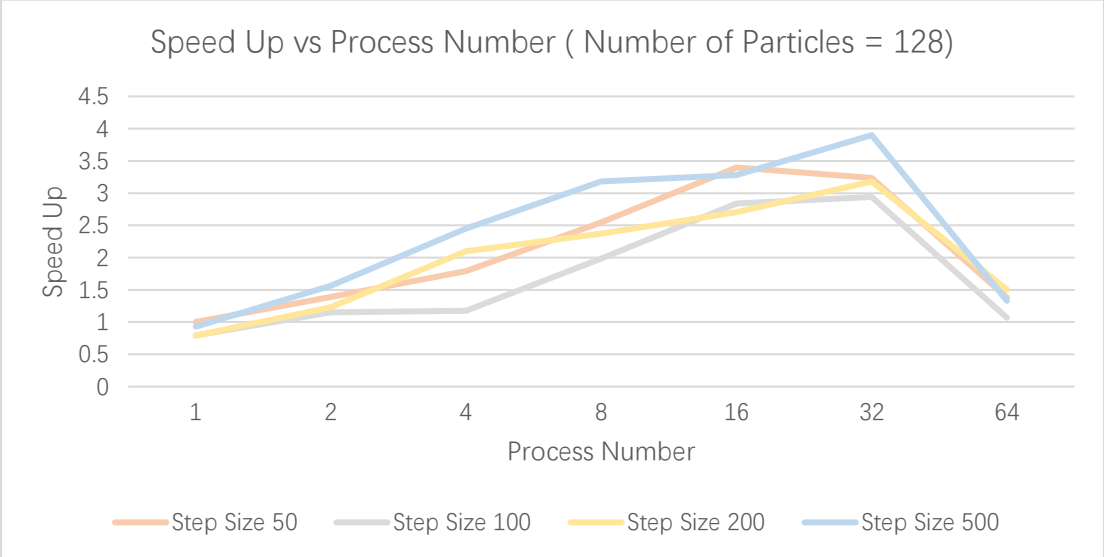
Number of time-steps	Time-step Length	Process Number	Execution Time	Speed Up
50	1	1	0.055018	0.95810462
		2	0.030253	1.742405712
		4	0.018567	2.839069317
		8	0.011891	4.433016567
		16	0.010215	5.160352423
		32	0.006167	8.547592022
		64	0.010909	4.832065267
100	1	1	0.108385	0.971379803
		2	0.056865	1.851455201
		4	0.036536	2.881623604
		8	0.021619	4.869929229
		16	0.016216	6.492538234
		32	0.010005	10.52303848
		64	0.013842	7.606054038
200	1	1	0.212449	0.992826514
		2	0.110894	1.902041589
		4	0.062166	3.392931828
		8	0.036900	5.716124661

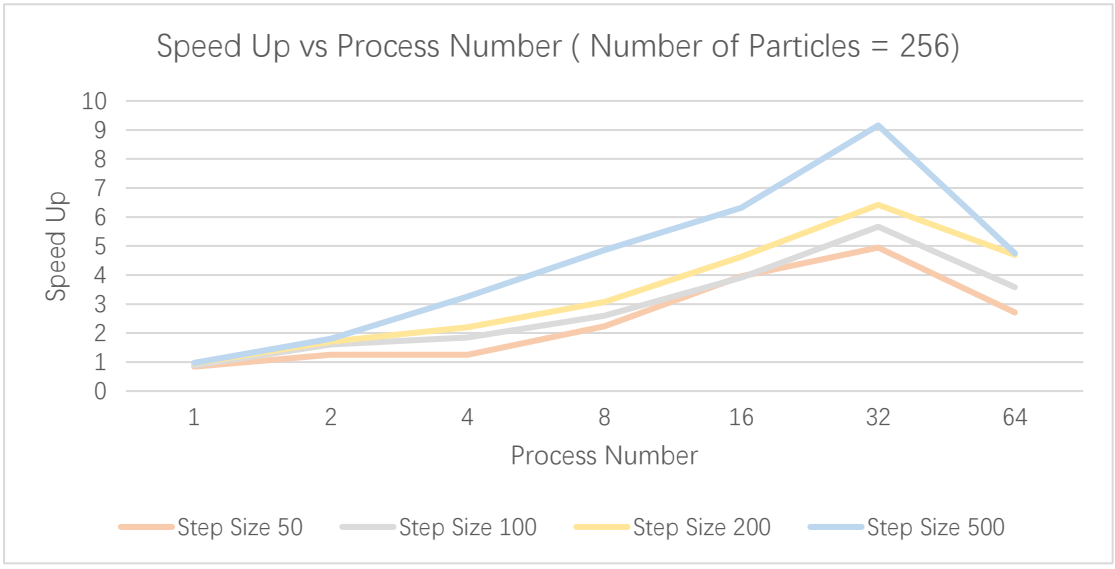
		16	0.024706	8.537399822
		32	0.017242	12.2332096
		64	0.019362	10.89376098
500	1	1	0.525246	1.004533114
		2	0.269942	1.95459395
		4	0.140341	3.759606957
		8	0.077370	6.819529533
		16	0.047743	11.05140021
		32	0.031721	16.63336591
		64	0.044985	11.7289541

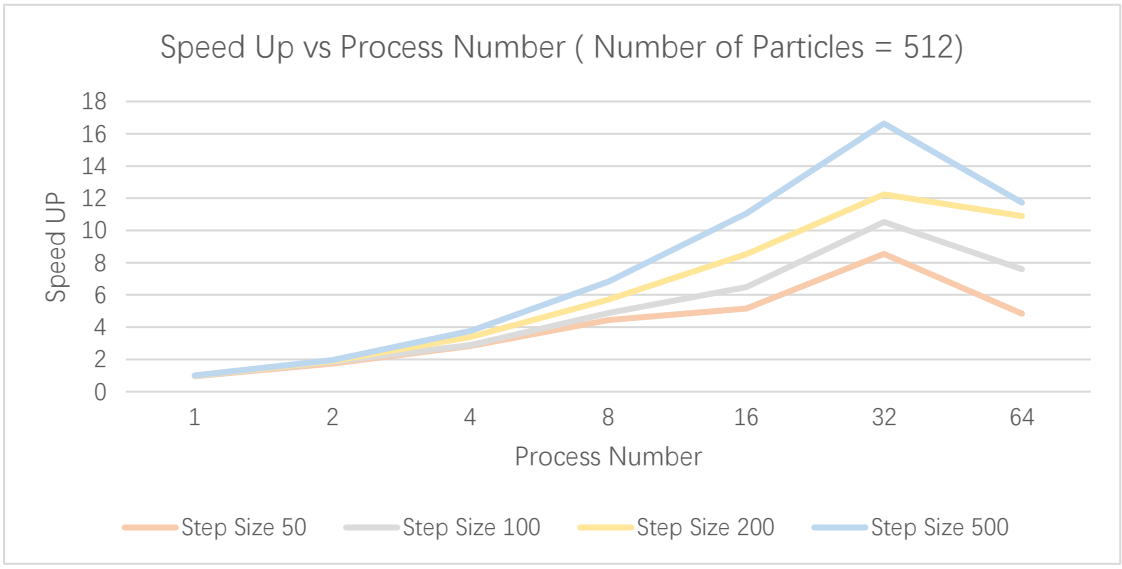
Number of Particles: 1024

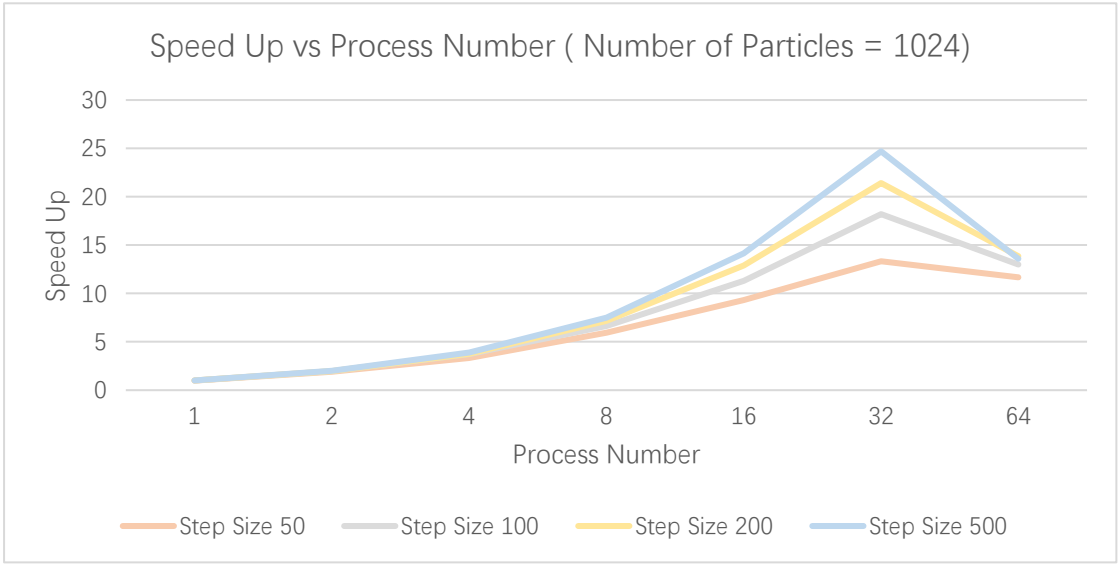
Number of time-steps	Time-step Length	Process Number	Execution Time	Speed Up
50	1	1	0.212267	0.98970636
		2	0.111119	1.890603767
		4	0.063165	3.325924167
		8	0.035397	5.935022742
		16	0.022572	9.307194755
		32	0.015761	13.32923038
		64	0.017999	11.67187066
100	1	1	0.421946	0.997874136
		2	0.214305	1.964718509
		4	0.113432	3.711906693
		8	0.063302	6.651432814
		16	0.037272	11.29665701
		32	0.023132	18.20201453
		64	0.032426	12.98491951
200	1	1	0.837665	1.005230014

		2	0.426263	1.97541424
		4	0.223327	3.770462147
		8	0.117644	7.157577097
		16	0.065292	12.89661827
		32	0.039321	21.41466392
		64	0.060747	13.86152403
500	1	1	2.090864	1.0068608
		2	1.057327	1.991067097
		4	0.538919	3.906355129
		8	0.280168	7.514095114
		16	0.148661	14.16113843
		32	0.085256	24.69279581
		64	0.155042	13.57831426




Speed Up vs Process Number (Number of Particles = 256)
Speed Up
10
9
8
7
6
5
4
3
2
1
0
1
2
4
8
16
32
64
Process Number
Step Size 50
Step Size 100
Step Size 200
Step Size 500


Speed Up vs Process Number (Number of Particles = 512)
Speed UP
18
16
14
12
10
8
6
4
2
0
1
2
4
8
16
32
64
Process Number
Step Size 50
Step Size 100
Step Size 200
Step Size 500


Speed Up vs Process Number (Number of Particles = 1024)
Speed Up
30
25
20
15
10
5
0
1
2
4
8
16
32
64
Process Number
Step Size 50
Step Size 100
Step Size 200
Step Size 500

The results show that the parallel MPI implementation achieves increasingly better performance as both the number of particles and the number of timesteps increase. For all particle sizes (128, 256, 512, and 1024), the execution time decreases as more processes are used, demonstrating that the computational workload is being successfully distributed among multiple MPI processes. Since the force calculation dominates the runtime and has a computational complexity of $O(n^2)$, larger particle counts provide more work per process, allowing the parallel implementation to utilize additional processors more effectively.

For the smallest problem size of 128 particles, the speedup is relatively limited, reaching only about 3–4× at 32 processes before declining significantly at 64 processes. This behavior indicates that communication and synchronization overhead become comparable to or even exceed the computation time. Because each process has only a small number of particles to compute, the cost of collective communication operations such as `MPI_Allgather` reduces the benefit of adding more processes. As a result, increasing the process count beyond 32 leads to diminishing returns and eventually a decrease in performance.

As the particle count increases to 256 and 512 particles, the scalability improves noticeably. The speedup curves rise more steadily with increasing process count, reaching approximately 9× for 256 particles and over 16× for 512 particles at 32 processes when using 500 timesteps. In these cases, the larger computational workload better amortizes the communication overhead. The force calculations occupy a greater proportion of the total execution time, making parallelization more effective and allowing the program to benefit from additional processing resources.

The best performance is observed for 1024 particles, where the speedup reaches approximately 24.7× using 32 processes and 500 timesteps. Similar trends are observed for the other timestep counts, with speedup consistently increasing as more processes are added up to 32. This demonstrates strong scalability for larger problem sizes

because each process performs a substantial amount of computation before synchronization occurs. The results indicate that the parallel implementation is most efficient when the ratio of computation to communication is high.

Another clear trend is that increasing the number of timesteps improves speedup for a fixed particle count. For example, with 1024 particles, the speedup at 32 processes increases from about 13.3× for 50 timesteps to approximately 24.7× for 500 timesteps. This occurs because longer simulations spend more time performing force calculations while the initialization and other fixed overheads remain relatively constant. Consequently, the parallel overhead becomes a smaller fraction of the total runtime, resulting in higher parallel efficiency.

Across all four graphs, the speedup generally peaks at 32 processes and then drops when the process count increases to 64. This decline is a common characteristic of strong scaling experiments. At 64 processes, each process is assigned fewer particles, reducing the amount of useful computation while communication costs continue to increase. The frequent `MPI_Allgather` operations required after every timestep become a larger portion of the total runtime, leading to reduced efficiency and lower speedup. For example, with 1024 particles and 500 timesteps, the speedup drops from approximately 24.7× at 32 processes to 13.6× at 64 processes.

Overall, the results demonstrate that the MPI-based parallel n-body implementation successfully accelerates the simulation and scales well for larger particle counts and longer simulations. The best performance is achieved with large workloads and around 32 processes, where the computation-to-communication ratio is most favorable. Beyond this point, communication overhead and synchronization costs begin to dominate, limiting further scalability and causing performance degradation at 64 processes. These observations are consistent with the expected behavior of an all-to-all n-body algorithm that relies on collective communication to maintain global particle position information.